

WEB DEVELOPMENT – III

Core Modules in Node.js

Session 5 — Study Notes

Topic: os, path, fs, crypto, and dns

Instructor: Love Porwal

K.R. Mangalam University — Sohna Road, Gurugram

1. What Are Core Modules?

A smartphone works the moment it's switched on: a camera, a file manager, Wi-Fi, a clock — none of these are downloaded from an app store, they simply come built in. Node.js follows the same idea. The moment it's installed, a set of ready-made tools is already available, with nothing to download and nothing to configure. These built-in tools are called core modules.

The mapping between the two is almost one-to-one, which makes the idea easy to hold onto:



Figure 1.1 — Familiar smartphone features, and the Node.js core module that plays the same role.

Formally, core modules are pre-built modules provided directly by Node.js to help developers perform common tasks — reading files, checking system information, generating secure random values — without installing a single additional package. They are available immediately after installing Node.js, and are imported the same way any module is, with `require()`, using just the module's name:

```
const os = require('os');
const fs = require('fs');
const path = require('path');
const crypto = require('crypto');
const dns = require('dns');
```

Six core modules come up constantly enough to be worth knowing well:

Module	What It's For
os	Platform, CPU, memory, and hostname information
path	Cross-platform file path operations — join, resolve, extname
fs	Read, write, update, and delete files and directories

Module	What It's For
dns	Resolve domain names to IP addresses, and back again
crypto	Hashing, HMAC, encryption, and secure random values
http	Create servers and make HTTP requests (covered in Session 2)

2. The OS Module

The `os` module answers questions about the machine Node.js is currently running on — not about the application, but about the physical or virtual computer underneath it. It can report the computer's hostname, its operating system, its CPU architecture, and how much memory is free versus how much exists in total.

This is genuinely useful in production, not just as a curiosity: a monitoring script can use it to check whether a server is close to running out of memory, or a support tool can print system details automatically when a bug report is filed.

```
const os = require('os');

console.log('Platform: ', os.platform()); // 'linux' | 'win32' | 'darwin'
console.log('Arch: ', os.arch()); // 'x64' | 'arm64'
console.log('Hostname: ', os.hostname());
console.log('Home Dir: ', os.homedir()); // /home/alice
console.log('Temp Dir: ', os.tmpdir()); // /tmp

// CPU information
const cpus = os.cpus();
console.log('CPU Count: ', cpus.length);
console.log('CPU Model: ', cpus[0].model);

// Memory (bytes converted to GB for readability)
const totalGB = (os.totalmem() / 1024 ** 3).toFixed(2);
const freeGB = (os.freemem() / 1024 ** 3).toFixed(2);
console.log(`Memory: ${freeGB} GB free of ${totalGB} GB total`);
```

3. The Path Module

The `path` module exists to solve a problem that looks trivial until it actually breaks something: Windows separates folders in a path with a backslash (`\`), while macOS and Linux use a forward slash (`/`). Code that builds a file path by simply gluing strings together with `+` will work on one operating system and quietly produce a broken path on the other. `path.join()` is written to always use whichever separator the current operating system actually expects.

```
const path = require('path');

// Join paths (handles OS separators automatically)
const full = path.join('/home', 'alice', 'projects', 'app.js');
console.log(full);           // /home/alice/projects/app.js

// Resolve to an absolute path (relative to the current working directory)
console.log(path.resolve('src', 'index.js'));

// Extract parts of a path
const file = '/home/alice/app.js';
console.log(path.basename(file));           // app.js
console.log(path.basename(file, '.js'));    // app (extension stripped)
console.log(path.dirname(file));            // /home/alice
console.log(path.extname(file));            // .js
```

`path.join('a', 'b')` returns `"a\\b"` on Windows but `"a/b"` on Unix-based systems — the exact same line of code, producing the correct separator on whichever machine it happens to run on. This is precisely what plain string concatenation cannot do, and it's why `path.join()` (not `+`) is the standard way to build any file path in Node.js.

4. The FS (File System) Module

The `fs` module is what lets Node.js reach outside of memory and actually touch the file system — reading a configuration file, saving a generated report, updating a log, or removing a temporary file. This is exactly the kind of capability browser JavaScript is deliberately denied, and precisely what makes Node.js useful as a backend runtime rather than just a scripting curiosity.

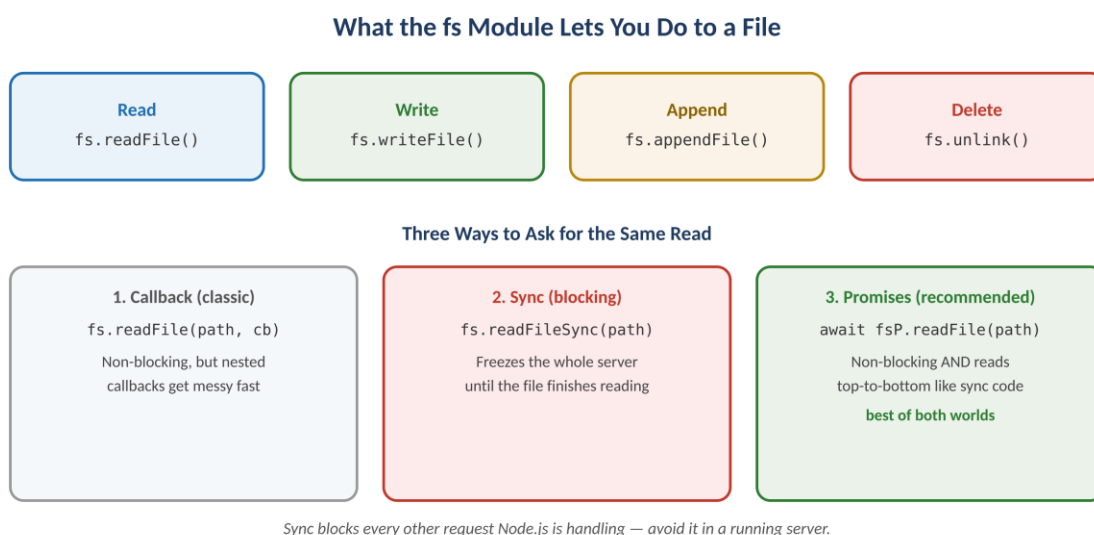


Figure 4.1 — The four things `fs` lets you do to a file, and three different ways to ask for a read.

Reading a File, Three Ways

Node.js offers three distinct styles for reading a file, and the differences between them matter more than they might first appear.

```
const fs = require('fs');

// 1. Async with a callback (the classic Node.js style)
fs.readFile('data.txt', 'utf8', (err, data) => {
  if (err) { console.error('Error:', err.message); return; }
  console.log('Content:', data);
});

// 2. Synchronous – blocks the entire event loop until it finishes!
try {
  const data = fs.readFileSync('data.txt', 'utf8');
  console.log('Content:', data);
} catch (err) {
  console.error('Error:', err.message);
}

// 3. Promises with async/await (recommended in modern code)
const fsP = require('fs/promises');

async function readData() {
  try {
    const data = await fsP.readFile('data.txt', 'utf8');
    console.log('Content:', data);
  } catch (err) {
    console.error('Error:', err.message);
  }
}

readData();
```

The synchronous version is the one to be genuinely careful with. Recall from Session 1 that Node.js handles many requests on a single thread by never sitting idle — `readFileSync` breaks that promise entirely: while it runs, nothing else Node.js is doing can proceed, including other users' requests on a live server. It has legitimate uses in short-lived scripts (a one-off setup script, a build tool), but should almost never appear inside code that handles live requests. The promise-based version, by contrast, keeps the non-blocking advantage of the callback style while reading top-to-bottom like ordinary synchronous code — which is why it's the recommended default in modern Node.js code.

Writing, Appending, and Deleting

`fs.writeFile()` creates a new file, or completely overwrites one that already exists — there is no merging or partial update involved.

```
const fs = require('fs');

// writeFile – creates the file, or OVERWRITES it if it already exists
fs.writeFile('output.txt', 'Hello World!\n', (err) => {
  if (err) throw err;
  console.log('File written!');
});

// Writing structured JSON data
const user = { name: 'Alice', role: 'admin' };
fs.writeFile('user.json', JSON.stringify(user, null, 2), (err) => {
  if (err) throw err;
  console.log('JSON file saved!');
});
```

When the goal is to add to a file without erasing what's already there — a log file being the classic example — `fs.appendFile()` is the correct tool instead:

```
const fs = require('fs');

// appendFile – adds to the existing content, without erasing it
const logEntry = `[${new Date().toISOString()}] Server started\n`;
fs.appendFile('server.log', logEntry, (err) => {
  if (err) throw err;
  console.log('Log updated!');
});

// Deleting a file
fs.unlink('temp.txt', (err) => {
  if (err && err.code !== 'ENOENT') throw err;
  console.log('File deleted!');
});
```

That `err.code !== 'ENOENT'` check in the delete example is a small but useful habit: `ENOENT` means the file "does not exist" already — which, for a delete operation, might not be worth treating as a real failure at all, since the end state (the file being gone) is exactly what was wanted anyway.

5. The Crypto Module

The `crypto` module provides the security primitives an application needs to handle sensitive data responsibly — hashing data, generating secure random values, and creating verifiable signatures. Without it, a common but serious mistake becomes tempting: storing passwords in plain text, or trusting incoming data with no way to

verify it hasn't been tampered with in transit. `crypto` exists precisely to make the correct, safer approach no harder than the careless one.

```
const crypto = require('crypto');

// SHA-256 hash (good for data integrity checks – not for raw passwords alone)
const hash = crypto
  .createHash('sha256')
  .update('my secret data')
  .digest('hex');
console.log('SHA-256:', hash);

// A UUID-style random, unique ID
const uuid = crypto.randomUUID();
console.log('UUID:', uuid);
```

Hash vs HMAC — Not the Same Tool

`createHash` and `createHmac` are easy to mistake for each other, but they solve different problems.

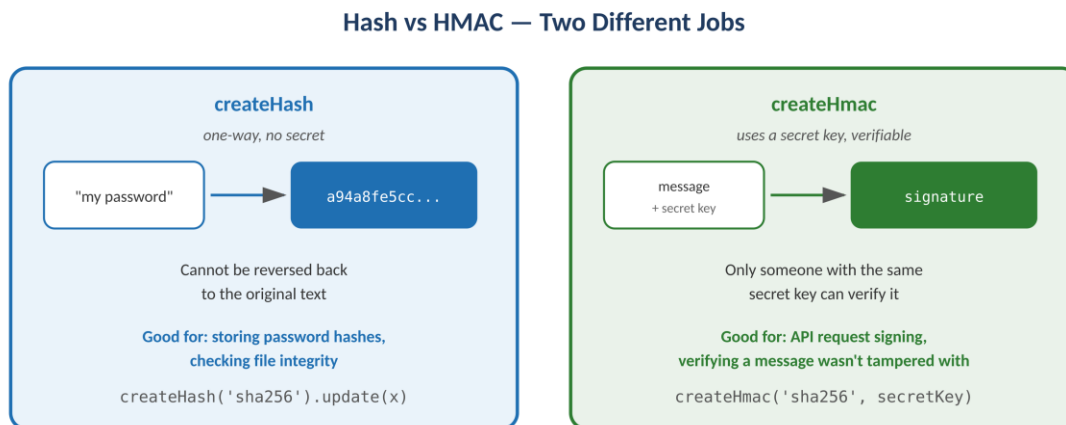


Figure 5.1 — A hash proves nothing about authorship; an HMAC does, because it needs a shared secret to verify.

`createHash` produces a one-way fingerprint of some data: the same input always produces the same hash, but there is no way to run the process backward and recover the original input from the hash alone. This is exactly the property wanted for storing password hashes or verifying that a downloaded file wasn't corrupted.

`createHmac` adds a secret key into that same process, producing a signature that only someone who also knows that secret key can independently reproduce and verify — which is what makes it suitable for signing API requests or confirming that a message genuinely came from a trusted sender and wasn't altered along the way. In short: a hash proves data integrity; an HMAC proves both integrity and authenticity.

6. The DNS Module

The dns module converts between domain names and IP addresses, the same lookup a browser performs invisibly every time a URL is typed in — except here it's callable directly from code. This matters wherever a network application needs to reason about servers by name: verifying that a domain actually resolves before trusting it, checking a mail server's records, or debugging why a particular hostname isn't reachable.

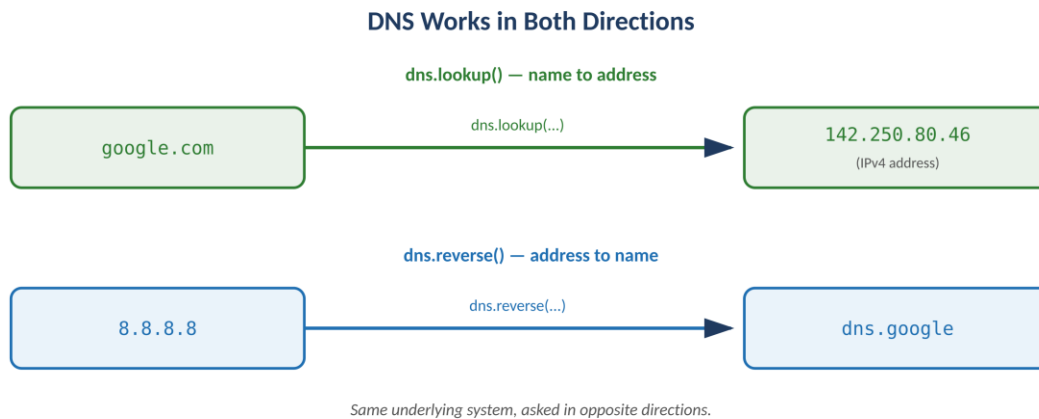


Figure 6.1 — Forward lookup finds an address from a name; reverse lookup finds a name from an address.

```
const dns = require('dns');

// Look up the IP address for a domain (uses the OS's resolver)
dns.lookup('google.com', (err, address, family) => {
  if (err) throw err;
  console.log('IP Address:', address);    // 142.250.80.46
  console.log('IPv' + family);           // IPv4
});

// Reverse lookup — IP address to hostname
dns.reverse('8.8.8.8', (err, hostnames) => {
  if (err) throw err;
  console.log('Hostname:', hostnames[0]); // dns.google
});
```

This shows up in real infrastructure more often than it might seem: network applications rely on DNS constantly, email servers check MX records before accepting mail for a domain, authentication systems verify domain ownership before issuing certificates, and hostname lookups are one of the first steps in debugging a connectivity problem.

7. Key Takeaways

To read current system memory, the correct module is `os` — specifically `os.totalmem()` and `os.freemem()`. `fs` handles files, `crypto` handles hashing and security, and `dns` handles domain resolution; none of them touch system memory information.

`path.join()` handles OS-specific path separators automatically — producing a backslash-joined path on Windows and a forward-slash-joined path on Unix-based systems from the exact same code. Plain string concatenation with `+` has no awareness of the operating system at all, which is exactly what breaks cross-platform compatibility.

`fs/promises` is generally preferable to the callback-based `fs` API because it enables `async/await`, which reads top-to-bottom like ordinary code instead of nesting callbacks inside callbacks — a pattern that becomes hard to follow as more asynchronous steps are chained together, often called "callback hell."

`createHash` produces a one-way result with no key involved, which makes it suitable for storing password hashes or checking file integrity. `createHmac` incorporates a secret key, producing a signature that can be verified only by someone who holds that same key — the right tool for signing API requests or confirming a message's authenticity, not just its contents.

8. Summary

- Core modules are pre-built tools that ship with Node.js itself — no npm install required, available the moment Node.js is installed.
- `os` reports information about the machine Node.js is running on: platform, CPU, memory, hostname.
- `path` builds and inspects file paths correctly across operating systems, avoiding the Windows-vs-Unix separator problem.
- `fs` reads, writes, appends to, and deletes files, available in callback, synchronous, and promise-based styles — promises are the modern default.
- `crypto` provides hashing (one-way, for integrity) and HMAC (keyed, for verifiable authenticity), along with secure random value generation.
- `dns` resolves domain names to IP addresses and back again, underpinning everything from browsers to mail servers to certificate issuance.

9. References

- Node.js Official Documentation — runtime, modules, APIs, file system, HTTP, and backend development concepts.
- W3Schools — Node.js Tutorial: beginner-friendly explanations with hands-on examples.
- freeCodeCamp — Node.js Course: free, project-based learning for Node.js, Express.js, and REST APIs.
- Node.js FS — the complete file system module API reference (nodejs.org/api/fs.html).